# MPTCP Extension Based on IP++

Chang Qing
Beijing IpPlusPlus Network Technology Co., Ltd.,Beijing 100195,China
Email:cq@ippp.xyz

**Abstract**

Multipath TCP (MPTCP) enables collaborative transmission across multiple network interfaces and delivers prominent advantages in transmission reliability and bandwidth utilization, making it a hot research and application topic in the networking field. Nevertheless, conventional IP protocols adopted by MPTCP feature a flat address design that fails to effectively describe the hierarchical structure of networks. Terminals are unable to control the forwarding paths of packets, which restricts MPTCP's multipath capability to mere multi-interface multiplexing. This limitation severely hinders MPTCP from unleashing its full potential. Designed to adapt to the inherent hierarchical structure of networks, IP++ natively supports path description as a fundamental protocol function and empowers terminals to independently manipulate packet transmission paths on demand. Leveraging the path control capabilities of IP++, this paper conducts an in-depth extension of MPTCP and elevates its performance to a new level.

**Keywords:** MPTCP; IP++; fixed-length address; variable-length address

# 1 Background of the Problem

## 1.1 Overview of MPTCP

Multipath TCP (MPTCP) [1] is an extension to standard TCP. It allows terminals to transmit and receive TCP packets over a single MPTCP connection via multiple network interfaces simultaneously. Compared to traditional TCP, MPTCP supports concurrent use of multiple transmission paths , thereby delivering performance benefits:

### 1.1.1 Link Aggregation

Transmission efficiency is improved by utilizing multiple paths concurrently. For instance, combining fixed-line and mobile networks accelerates file delivery.

### 1.1.2 Optimal Path Selection

The protocol can select the optimal path for data transmission according to metrics such as latency, loss rate, cost and bandwidth.

### 1.1.3 Seamless Handover

When one path fails, MPTCP can continue data transmission over other available paths without interrupting communication.

## 1.2 Evolution of Traditional IP Protocols

Major traditional IP protocols include IPv4 and IPv6 [2, 3]. IPv4 uses 32-bit addresses [4], and the problem of address exhaustion has long been prominent. To address this issue, Network Address Translation (NAT) was introduced [5]. Essentially, NAT enables multiple internal network devices to share a limited number

of public IP addresses. However, it breaks the end-to-end principle and prevents extranet directly accessing intranet devices.

IPv6 adopts 128-bit addresses and fundamentally resolves the address shortage problem [6, 7, 8]. Even so, IPv6 has two critical drawbacks. First, it suffers from poor interoperability with IPv4, making seamless migration impossible. Upgrading existing IPv4 devices and network infrastructures incurs enormous costs and hinders large-scale deployment. Second, its long addresses bring difficulties in configuration, management and memorization, resulting in poor usability. Such issues make IPv6 inapplicable in certain scenarios such as internal networks. For these reasons, IPv6 cannot completely replace IPv4 or serve as a unified next-generation IP protocol.

## 1.3 Fixed-length vs. Variable-length Addresses

Variable-length address schemes were considered in the initial design of IP [9]. The debate between fixed-length and variable-length address architectures has run through the entire evolution of IP protocols. A full variable-length address architecture has always been the ultimate vision of IP designers [10, 11], while fixed-length addresses have been widely deployed in commercial products due to lower hardware overhead and simpler implementation.

All mainstream commercial IP protocols are compromises between theoretical design ideals and practical engineering constraints. IPv4 purely employs fixed-length addresses and gradually incorporates hierarchical addressing mechanisms during evolution, leading to the classification of public and private networks as well as NAT. Although IPv6 introduces variable-length prefixes, it still adopts a fixed-length address architecture. The failure to fully implement variable-length addressing is the root cause of various inherent defects of IPv6.

With the substantial advancement of network hardware performance and software development technologies, technical bottlenecks restricting the engineering deployment of variable-length address schemes have been eliminated. The condition for large-scale adoption of variable-length address architecture are now mature.

## 1.4 Limitations of Traditional IP Protocols

The aforementioned defects of IPv4 and IPv6 are summarized in this section. Both protocols adopt fixed-length address architectures, and their common drawbacks stem from the inherent limitations of this design:

### 1.4.1 Non-scalable Address Length

The fixed address length inevitably brings corresponding weaknesses. The short addresses of IPv4 lead to severe address exhaustion, while the long addresses of IPv6 increase the complexity of device configuration, operation, maintenance and manual management, creating major obstacles to popularization.

### 1.4.2 Poor Scalability

Network topologies are non-uniform and dynamically evolving, and different network branches have distinct path lengths and hierarchy levels that cannot be standardized by centralized rules. Using fixed-length fields to encode addresses for variable topologies will eventually result in insufficient address space and forced architecture reconstruction, which conflicts with the long-term evolution requirements of the global network.

### 1.4.3 Lack of Network Structural Information in Addresses

Flat addresses in traditional IP protocols cannot carry network hierarchy and topology information, and thus fail to support multipath applications. Restricted by this, current mainstream multipath transport technologies including MPTCP and MPQUIC can only realize multi-interface multiplexing, rather than genuine multipath scheduling at the protocol layer.

Segment Routing technology mainly includes SR-MPLS and SRv6, which can achieve path control by specifying routers along the path. However, based on flat addresses, it can only express flat paths (linear node sequences), unable to represent network hierarchy and unable to achieve "address as path". Primarily designed for forwarding acceleration, Segment Routing is not suitable for MPTCP scenarios.

# 2 Advantages of IP++ in Path Control

## 2.1 Overview of IP++

While the division of public and private networks in IPv4 reflects preliminary hierarchical design ideas, it does not form a systematic multi-level variable-length architecture. The core design of IP++ is to introduce a comprehensive multi-level variable-length address scheme into the network layer, building an addressing system fully matching the hierarchical network topology [12, 13].

In the IP++ architecture, the Internet is divided into multiple hierarchical levels. Each level and its subnets own independent address spaces, which are defined as unitnets. Each unitnet has an address space equivalent to that of IPv4. The overall Internet presents a tree topology: the top layer is the root public unitnet, with sub-unitnets attached level by level to form a complete tree structure.

Both unitnets and connected devices are assigned explicit hierarchy levels, which can be expressed in two ways: absolute level and relative level. An absolute level is prefixed with "/", represented by a non-negative integer. The public network is defined as level /0, with subsequent sub-levels numbered /1, /2, and so on. A relative level is prefixed with ".", represented by an integer. The current unitnet is level .0, its parent unitnet is level .-1, the grandparent unitnet is level .-2, and so on upwards.

An IP++ address is formed by concatenating address segments of each hierarchy level, similar to communication addresses or file paths. A complete IP++ address starts with the initial hierarchy level, e.g., /1#192.168.1.10. IP++ also supports relative addresses, which work like relative paths in file systems. For example:

.0#192.168.1.10 refers to the address 192.168.1.10 within the current unitnet.

.-1#10.66.7.250 refers to the address 10.66.7.250 in the parent unitnet of the current unitnet.

## 2.2 Path Control Capabilities of IP++

IP++ divides internet into unitnets according to inherent network hierarchies, forming an approximate tree structure. An IP++ address inherently describes the packet forwarding path across unitnets, realizing the design of address as path. Although the address does not contain detailed forwarding rules inside a single unitnet, it fully defines the inter-unitnet routing.

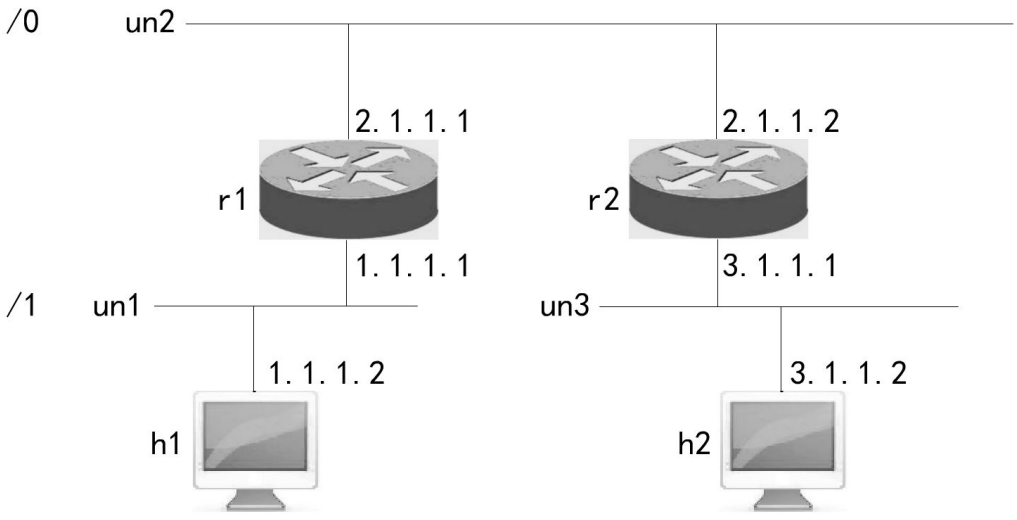


Figure 1: Example of a Simple Network Topology

As shown in Figure 1, when host h1 sends packets to host h2, the destination address is .-1#2.1.1.2-3.1.1.2, which explicitly defines the transmission path:

1.Packets are forwarded upward to the parent unitnet un2 (the system default gateway 1.1.1.1 is used if no inter-unitnet router is specified).

2.The packet reaches node 2.1.1.2 inside un2.

3.The packet enters the sub-unitnet un3 attached to router r2 and finally arrives at host h2 with address 3.1.1.2.

For upward transmission in the hierarchy, the default gateway is adopted by default. Users can manually specify routers via absolute paths or relative paths. Braces are used to denote custom routers for absolute addresses. Taking Figure 1 as an example, to specify the upper router r1 explicitly, the address can be written in the following forms:

- .-2.1.1.1(1.1.1.1)#2.1.1.2-3.1.1.2
- .-2.1.1.1#2.1.1.2-3.1.1.2
- .-(1.1.1.1)#2.1.1.2-3.1.1.2

For an absolute address such as /#2.1.1.2-3.1.1.2, the custom router is specified as:
/-2.1.1.1(1.1.1.1)#2.1.1.2-3.1.1.2

# 3 Deployment of IP++ in MPTCP

## 3.1 Multipath Control with IP++

In a pure tree topology, there is only one single path between any two nodes. However, real-world networks are commonly equipped with multipath topologies, as illustrated in Figure 2 and Figure 3.

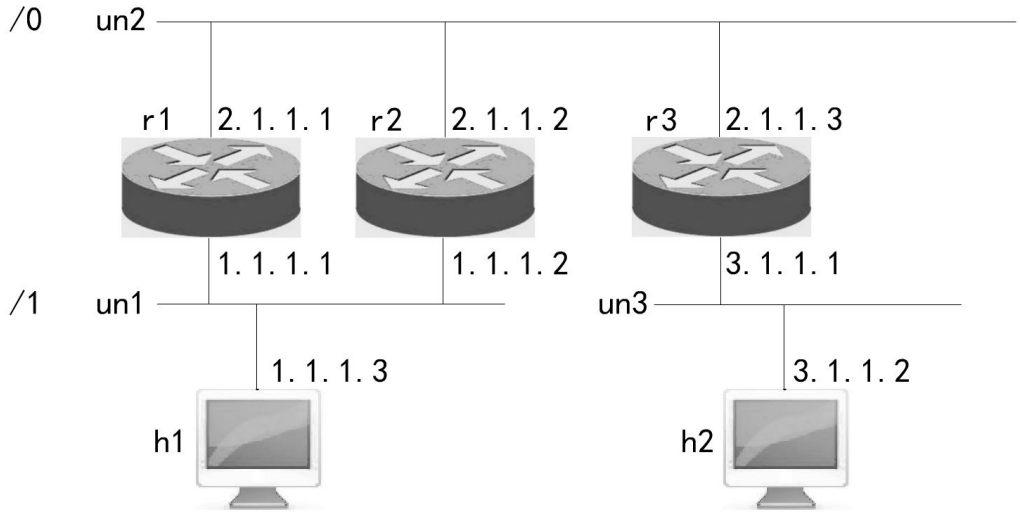


Figure 2: Example 1 of Multipath Network Topology

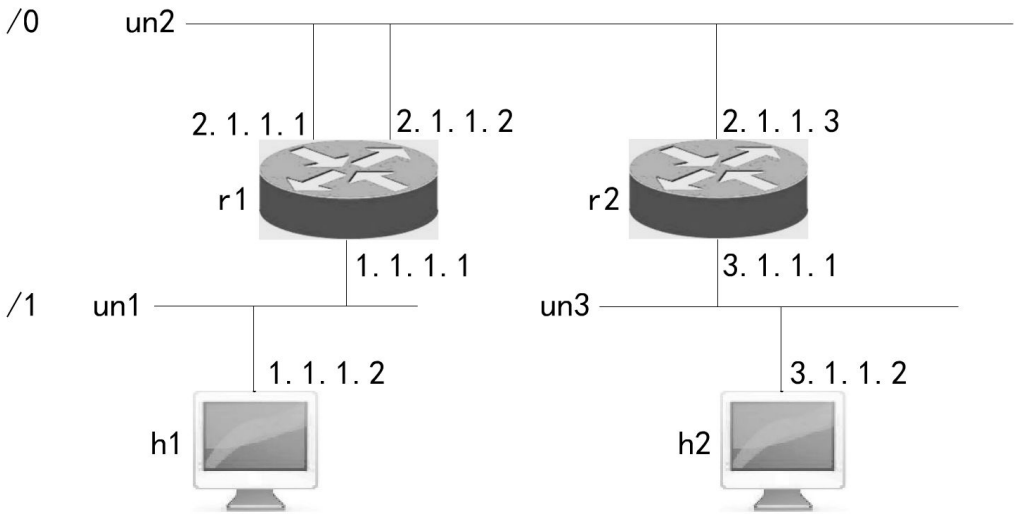


Figure 3: Example 2 of Multipath Network Topology

Multiple alternative paths exist between host h1 and host h2 in these scenarios. With IP++, users can explicitly select a specific path via address configuration. For the topology in Figure 2, the address .-(1.1.1.1)#2.1.1.2-3.1.1.2 can be used to designate a dedicated transmission path.

This is clearly different from traditional IP protocols, where packet routing depends on network configuration and terminals have no control over it.

### 3.2 Path Management in MPTCP

Traditional MPTCP manages multiple network interfaces via the mptcp subcommand of the ip tool. In the IP++ environment, the dedicated ippp command is adopted for path management.

For the network topology in Figure 2, use the following command to add path:

ippp mptcp path add .-[(1.1.1.1), (1.1.1.2)]#*

To adopt system default routers without explicit path specification:

ippp mptcp path add .-[(_), (1.1.1.2)]#*

For the topology in Figure 3, the corresponding command is:

ippp mptcp path add .-[2.1.1.1, 2.1.1.2]#*

Or use default routers:

ippp mptcp path add .-[_, _]#*

## 4 Conclusion

IP++ delivers superior path control capabilities that cannot be achieved by traditional IP protocols. When integrated with MPTCP, it enables genuine multipath transmission and greatly expands the application scope of MPTCP. This innovative application further verifies the technical advantages of IP++ and its inevitability to replace traditional IP protocols.

Furthermore, the multipath extension scheme based on IP++ proposed in this paper is also applicable to other multipath transport protocols such as MPQUIC [14] and MPUDP, and thus possesses broad promotion and application value.

## References


[1] A FORD, C RAICIU, M HANDLEY, et al. Rfc 6824: Tcp extensions for multipath operation with multiple addresses (mptcp). S, 2013.

[2] X R Xie. *Computer Networks*. Publishing House of Electronics Industry, 8th edition, 2021.

[3] K R FALL, W R STEVENS, and G R WRIGHT. *TCP/IP Illustrated*. China Machine Press, 2016.

[4] J POSTEL. Rfc 791: Internet protocol. S, 1981.

[5] P SRISURESH and M HOLDREGE. Rfc 2663: Ip network address translator (nat) terminology and considerations. S, 1999.

[6] S DEERING and R HINDEN. Rfc 8200: Internet protocol, version 6 (ipv6) specification. S, 2017.

[7] R HINDEN and S DEERING. Rfc 4291: Ip version 6 addressing architecture. S, 2006.

[8] D D CLARK, L CHAPIN, V G CERF, et al. Towards the future internet architecture, 1991.

[9] D D CLARK. *Designing an Internet (Information Policy)*. MIT Press, 2018.

[10] J N CHIAPPA. Why topologically sensitive names are inherently variable length, 1994.

[11] C Huitema. Flexible ip: An adaptable ip address structure. IETF 109, 2021.

[12] Ltd. Beijing IpPlusPlus Network Technology Co. Ip++ official website. EB/OL.

[13] Q Chang. Complete ip protocol. EB/OL.

[14] Y M LIU, Y F MA, Q DE CONINCK, et al. Multipath extension for quic. S, 2026. draft-ietf-quic-multipath-20.